High-Performance Computer Architecture
Chapter Practice Question Answer Sheet

ECE 4100/6100

Spring 2026

Contents

Scope **1**

1 Chapter 1: Introduction **1**

2 Chapter 2: Performance Evaluation **2**

3 Chapter 3: Cache Memories **3**

4 Chapter 4: Pipelining **5**

5 Chapter 5: Integrated Synthesis and Cache Simulation Concepts **6**

6 Chapter 6: Dynamic ILP and Out-of-Order Execution **7**

7 Chapter 7: Compiler-Scheduled ILP **8**

8 Chapter 8: Multiprocessors and Coherence **9**

9 Chapter 9: Synchronization, Transactions, and Consistency **12**

10 Chapter 10: Parallel Interconnection Networks **14**

Scope

This document answers the numbered practice questions at the end of Chapters 1–10 of the review textbook. It is intentionally separate from the main text so it can be distributed or withheld independently.

1 Chapter 1: Introduction

1. Packed-word SIMD primarily exploits parallelism. A cache primarily exploits speculation without prediction by keeping nearby or recently used data before proving it will be needed. A branch predictor supports speculation with prediction. Out-of-order execution with branch prediction combines instruction-level parallelism with speculation directed by prediction.

2. A dense image convolution engine is usually SIMD because the same operation is applied across many pixels. A cloud-service cluster is MIMD because different processors execute independent instruction streams for different requests. A small thermostat controller is SISD in the usual scalar embedded-control model. A brute-force hash search implemented as many independent workers is MIMD; a lockstep version of the same arithmetic could instead be SIMD-like.

3. Pipelining is orthogonal to Flynn's taxonomy because it overlaps stages over time rather than changing the number of instruction streams or data streams. A five-stage scalar integer pipeline is a pipelined SISD example. A SIMD vector unit can pipeline its lanes or arithmetic stages, and each core in a MIMD multicore can also contain an internal pipeline.

4. Serial vector execution needs

$$nk = 100 \cdot 6 = 600 \text{ cycles.}$$

Pipelined vector execution needs approximately

$$k + n - 1 = 6 + 100 - 1 = 105 \text{ cycles.}$$

Chaining lets early elements produced by the first vector operation feed the dependent vector pipeline immediately, instead of waiting for the entire first vector to finish.

5. Packed-word SIMD fixes the amount of parallel work by register width and lane width, such as four 32-bit lanes in a 128-bit register. Classic pipelined vector execution streams a vector of many elements through functional-unit stages; the vector length, not just the register width, determines how many elements flow through the pipeline.

2 Chapter 2: Performance Evaluation

1.

$$T_A = \frac{1.2 \times 10^9 \cdot 1.5}{2.4 \times 10^9} = 0.75 \text{ s}, \qquad T_B = \frac{0.9 \times 10^9 \cdot 1.9}{3.0 \times 10^9} = 0.57 \text{ s}.$$

Machine B is faster. Its speedup over A is

$$\frac{0.75}{0.57} \approx 1.32.$$

2. With the same instruction count,

$$\text{speedup}_{K/J} = \frac{3.0/5.1}{3.7/5.5} \approx 0.875.$$

K is therefore slower than J. For G,

$$\text{speedup}_{G/J} = \frac{3.0/5.1}{(1.5)(3.2)/6.1} \approx 0.748.$$

G is also slower than J for this workload.

3.

$$T_{avg} = 0.50(4) + 0.30(9) + 0.20(6) = 5.9 \text{ s}.$$

The same arithmetic mean is inappropriate for throughputs because throughputs are rates. For equal work, total time is proportional to the reciprocal of throughput, so a harmonic mean or an explicit total-work divided by total-time calculation is needed.

4. Amdahl's Law gives
$$S = \frac{1}{(1 - 0.30) + 0.30/4} = \frac{1}{0.775} \approx 1.29.$$
If the branch-recovery portion became infinitely fast, the maximum speedup would be
$$\frac{1}{0.70} \approx 1.43.$$

5.
$$S = \frac{1}{0.80 + 0.20/4} = \frac{1}{0.85} \approx 1.18.$$
More parallel hardware has diminishing returns because the unaccelerated 80% remains. Even an infinitely fast accelerator could only reach $1/0.80 = 1.25$.

6. Gustafson's Law gives
$$S_G = N - \alpha(N - 1) = 16 - 0.10(15) = 14.5.$$
This does not contradict Amdahl's Law because Gustafson's Law assumes a scaled problem size, while Amdahl's Law analyzes a fixed-size workload.

7. Equal-work runtimes can be summarized with an arithmetic mean of times. Equal-work throughputs should use a harmonic mean or an explicit total-work/total-time calculation. A production workload mix with known fractions should use a weighted mean of the relevant quantity, usually weighted runtime if the fractions describe time or workload share.

8. Server 1:
$$\frac{1.25}{4000} = 0.0003125 \text{ speedup/dollar}, \qquad \frac{4000}{1.25} = 3200 \text{ dollars per unit speedup.}$$
Server 2:
$$\frac{1.55}{5400} \approx 0.000287, \qquad \frac{5400}{1.55} \approx 3484.$$
Server 1 is better under both speedup-per-dollar and cost-per-unit-speedup, although Server 2 has the larger absolute speedup.

3 Chapter 3: Cache Memories

1. 32 KiB $= 2^{15}$ bytes, 64 B $= 2^6$ bytes, and 4-way associativity is 2^2 ways.
$$\text{sets} = 2^{15-6-2} = 2^7 = 128.$$
Offset bits $= 6$, index bits $= 7$, and tag bits $= 48 - 6 - 7 = 35$. There are $2^{15}/2^6 = 512$ lines. With tag plus valid plus dirty, each line stores $35 + 2 = 37$ bits, so the tag-store total is
$$512 \cdot 37 = 18{,}944 \text{ bits} = 2{,}368 \text{ bytes.}$$

2. A victim cache primarily targets miss ratio, especially conflict misses. A write buffer primarily reduces processor-visible write stall or miss-penalty effects. Increasing associativity primarily targets miss ratio. Increasing block size primarily targets miss ratio by exploiting spatial locality, though it can also increase miss penalty. Critical-word-first with early restart primarily targets miss penalty.

3. Early restart lets the processor resume as soon as the requested word arrives. Critical-word-first changes the transfer order so the requested word is returned first. Critical-word-first alone does not reduce processor-visible miss penalty if the processor still waits for the entire block before restarting.

4. The block contains $32/8 = 4$ transfers. Baseline full-block repair has miss penalty $4 \cdot 70 = 280$ ns, so

$$AAT = 10 + 0.15(280) = 52 \text{ ns}.$$

Early restart with a uniformly distributed critical word has average visible miss penalty

$$\frac{70 + 140 + 210 + 280}{4} = 175 \text{ ns},$$

so

$$AAT = 10 + 0.15(175) = 36.25 \text{ ns}.$$

Critical-word-first with early restart makes the visible penalty about 70 ns, so

$$AAT = 10 + 0.15(70) = 20.5 \text{ ns}.$$

5. For $(C, B, S) = (6, 4, 2)$, there is one set with four ways, so the tag is the block number after removing the 4 offset bits.

Access	Tag	Outcome
`0x589e`	`0x589`	miss
`0xa7a1`	`0xa7a`	miss
`0x3767`	`0x376`	miss
`0xcce2`	`0xcce`	miss
`0xa7aa`	`0xa7a`	hit
`0xff4d`	`0xff4`	miss, evict `0x589`
`0x376c`	`0x376`	hit
`0x5892`	`0x589`	miss, evict `0xcce`

Final LRU-to-MRU tag order is `0xa7a`, `0xff4`, `0x376`, `0x589`.

6. Starting with way 0 least recent and way 3 most recent, the access sequence $2 \to 0 \to 3 \to 1 \to 2$ leaves the final LRU-to-MRU order

$$0,\ 3,\ 1,\ 2.$$

Thus way 0 is least recently used. The final off-diagonal recency matrix is

	0	1	2	3
0	–	0	0	0
1	1	–	0	1
2	1	1	–	1
3	1	0	0	–

where $M[i][j] = 1$ means way i is more recent than way j.

7. The VIPT constraint is $C - S \le P$, where $P = \log_2(4 \text{ KiB}) = 12$. Since $C = \log_2(64 \text{ KiB}) = 16$, the cache needs

$$S \ge C - P = 4.$$

The minimum associativity is therefore $2^4 = 16$-way.

8. A small victim cache is the more natural first fix. The pattern is a direct-mapped conflict between a small number of blocks, so a victim cache can hold the recently evicted block. A simple +1 prefetcher is aimed at sequential access and does not solve the placement conflict.

9. $$3b^2 \cdot 8 \le 24 \text{ KiB} = 24{,}576 \quad \Rightarrow \quad 24b^2 \le 24{,}576 \quad \Rightarrow \quad b^2 \le 1024.$$

   The largest integer tile size is $b = 32$.

10. Tiling changes the order of memory references so the program reuses a cache-sized working set before moving to the next tile. The arithmetic and total logical data accesses are the same, but the temporal locality seen by the cache is better, so fewer useful blocks are evicted before reuse.

11. $$AAT_{L2} = 8 + 0.25(60) = 23 \text{ cycles.}$$

    Then

    $$AAT_{L1} = 1 + 0.08(23) = 2.84 \text{ cycles.}$$

12. The first write to A creates a dirty A line. The write to B creates a dirty B line. The second write to A keeps A dirty and usually refreshes its replacement state. If A and B fill the set, then the later access to C evicts the least-recently used dirty line, which is B under ordinary LRU, and that eviction causes a writeback.

4 Chapter 4: Pipelining

1. The maximum ideal speedup is 5 if the pipelined clock period is unchanged and the ideal pipelined CPI is 1. With 25% of instructions taking one extra stall cycle,

   $$CPI = 1 + 0.25(1) = 1.25.$$

   Relative to the original $CPI = 5$, this corresponds to a speedup of $5/1.25 = 4$.

2. $$CPI = 1 + 0.05 + 0.12 + 0.15(2) = 1.47.$$

3. RAW dependences are: load to add on $R1$, add to sub on $R3$, and sub to store on $R5$. There are no register WAR or WAW dependences in this sequence. The most important forwarding paths are the load-result-to-EX path for the load-use pair and the ALU-result forwarding paths for the add-to-sub and sub-to-store chain.

4. Fetch overlapping with data-memory access implies separate instruction/data memories or enough memory ports. An ALU consumer reaching EX before the producer reaches WB implies forwarding or bypassing. A decode-stage consumer reading a value in the same cycle an older instruction writes it back needs write-first/read-later register-file timing or an equivalent WB-to-ID bypass.

5. The BTB predicts the target address. The RAS predicts return targets by tracking call-return nesting. The direction predictor predicts taken versus not taken. A BTB remains useful even for unconditional or trivially taken branches because fetch still needs the target early.

6. The call at `0x4000` pushes return address `0x4004` onto the RAS and creates or refreshes a BTB target entry for `0x4A00`. The taken conditional branch at `0x4A00` creates or refreshes a BTB entry for target `0x4AF0`. The return at `0x4AF8` uses and pops the RAS entry, predicting `0x4004`. After the return, the RAS is empty. Both `0x4000` and `0x4A00` select BTB index 0. Under the specified direct-mapped convention, the conditional branch replaces the call's tagged entry; the return does not allocate an entry. The final valid index-0 entry identifies branch `0x4A00` and target `0x4AF0`.

7. Using rightmost-bit-newest GHR shifting:

Step	$PC[2]$	GHR_T	Ctr_T	$Pred.$	Ctr_{T+1}	GHR_{T+1}
$(0x2004, T)$	1	`01`	`01`	N	`10`	`11`
$(0x2008, N)$	0	`11`	`00`	N	`00`	`10`
$(0x2004, T)$	1	`10`	`00`	N	`01`	`01`
$(0x200C, T)$	1	`01`	`10`	T	`11`	`11`

8. GSelect combines a global history register with PC bits to index the counter table directly. Yeh/Patt first uses PC bits to select that branch's local history register; that local history then selects a pattern-table counter. Yeh/Patt can distinguish two static branches that have different repeating local patterns but would otherwise alias under one shared global history.

9. The claim is not necessarily correct. The local history value `00` selects a pattern-table entry, but the state of that selected counter determines the prediction. If the selected counter is initialized to a taken state, the first prediction is taken even though the local history is `00`.

10. The history `1010` maps to $x = [+1, -1, +1, -1]$.

$$y = 1 + 2(+1) + (-1)(-1) + 0(+1) + 1(-1) = 3.$$

The prediction is taken. The actual outcome is not taken, so training occurs; also $\theta = \lfloor 1.93(4) + 14 \rfloor = 21$. With $t = -1$, the updated weights are

$$[0, 1, 0, -1, 2].$$

Shifting not taken into the rightmost position changes the GHR from `1010` to `0100`.

5 Chapter 5: Integrated Synthesis and Cache Simulation Concepts

1.

$$AAT_{L2} = 7 + 0.20(80) = 23, \qquad AAT_{L1} = 1 + 0.10(23) = 3.3.$$

The L1 miss penalty is the lower-level average because an L1 miss may hit in L2 or may miss again and continue to DRAM.

2. The 70 prefetch hits show useful prefetched lines. The 25 evicted-before-use lines show wasted bandwidth, cache space, and possible pollution. Before calling the prefetcher a net win, check demand misses avoided, extra traffic, pollution effects, timeliness, and net CPI or AAT change.

3. With write-through, write-no-allocate behavior, many lower-level writes are pass-through stores or writebacks rather than demand reads that allocate a useful line. A miss ratio over all accesses can therefore mix unlike events and obscure the behavior of read misses, which are usually the accesses that determine demand miss penalty.

4. The better design depends on

$$AAT = HT + MR \cdot MP.$$

A lower hit time can be outweighed by a higher miss ratio if misses are expensive, while a lower miss ratio can be outweighed by a slower hit path if misses are rare.

5. Read-miss repair and writeback traffic compete for lower-level queues, buses, replacement choices, and sometimes cache state. Changing service order can change which line is present when a later access arrives, which counters are incremented, and whether a later request sees a hit, miss, writeback, or intervention.

6. Load-miss stalls contribute

$$0.30(0.05)(20) = 0.30.$$

Branch stalls contribute

$$0.18(0.10)(2) = 0.036.$$

Total CPI is

$$1 + 0.30 + 0.036 = 1.336.$$

Load misses contribute more.

6 Chapter 6: Dynamic ILP and Out-of-Order Execution

1. The maximum IPC is

$$\min(6, 4, 3, 5) = 3.$$

The three function units create the bound.

2. Counting height as the number of edges on the longest path to exit G:

$$G = 0, \quad E = 1, \quad F = 1, \quad C = 2, \quad D = 2, \quad A = 3, \quad B = 3.$$

If the convention counts nodes instead of edges, add one to each value.

3. One consistent FICO timing is:

Instr.	Dispatch	Fire	Complete
A : `LW`	1	2	3
B : `ADD`	1	4	4
C : `ADD`	2	5	5
D : `ADD`	2	6	6

These cycles use the question's explicit next-cycle result-availability convention. B waits for A, and C cannot pass the blocked B even though C is independent. With one ADD unit, B, C, and D use consecutive fire cycles 4, 5, and 6. A second ADD unit lets B and C fire together in cycle 4 and D fire in cycle 5, but it cannot eliminate the earlier wait for A or let C bypass B while B is blocked.

4. For `LW R1,0(R3)`, the source $R3$ comes from the register file and the destination is the first ROB producer for $R1$. For `ADD R3,R2,R1`, $R2$ comes from the register file and $R1$ waits on the load's ROB tag. For `ADD R1,R3,R2`, $R3$ waits on the second instruction's ROB tag and $R2$ comes from the register file.

5. Immediately after dispatch:

Instr.	Destination tag	Ready sources	Waiting sources
A : `LW R1,0(R3)`	$A/R1$	$R3$	none
B : `ADD R3,R2,R1`	$B/R3$	$R2$	A
C : `ADD R1,R3,R2`	$C/R1$	$R2$	B

6. The dependence chain forces the earliest completion order A, then B, then C. Results also become architecturally visible in program order A, B, then C, because retirement is in order even when execution machinery is out of order.

7. Dispatch changes the RAT mapping for `R1` from `P10` to `P24`. `P10` cannot be freed immediately because older in-flight instructions may still need the old mapping, and recovery from a misprediction or exception may still need the previous precise state.

8. A CDB broadcast makes a computed value visible to waiting microarchitectural structures and marks a tag ready. Retirement updates precise architectural state in program order. The distinction is essential because out-of-order completion must not expose younger results architecturally before older instructions are known to be safe.

9. (a) True: renaming removes WAR and WAW name dependences, but true RAW flow dependences remain. (b) False: a value can be present in a future file or physical register before it has retired into precise architectural state. (c) True: fetched instructions include wrong-path work, so fetched-instruction IPC can overstate useful architectural progress.

10. Architecturally meaningful IPC uses retired instructions:

$$IPC = \frac{1200}{1000} = 1.2.$$

The 400-instruction fetched/retired gap suggests wrong-path or otherwise squashed speculative work. The 90 ROB-blocked dispatch cycles suggest that the reorder buffer filled or could not drain fast enough, often because older long-latency instructions delayed retirement.

7 Chapter 7: Compiler-Scheduled ILP

1. Let the instructions be I_1 = `LW R1,X`, I_2 = `ADD R2,R1,R3`, I_3 = `ADD R1,R4,R5`, and I_4 = `MUL R6,R2,R1`. Pure/RAW dependences are $I_1 \to I_2$ on $R1$, $I_2 \to I_4$ on $R2$, and $I_3 \to I_4$ on $R1$. The anti-dependence is $I_2 \to I_3$ on $R1$, because I_2 reads the old $R1$ before I_3 overwrites it. The output dependence is $I_1 \to I_3$ on $R1$.

2. Using load latency 2, add latency 1, and multiply latency 3, while preserving all original dependences, the heights are

$$I_4 = 3, \quad I_3 = 4, \quad I_2 = 5, \quad I_1 = 7.$$

Descending order is I_1, I_2, I_3, I_4. If false dependences are removed by renaming, then $I_1 = 6$, $I_2 = 4$, $I_3 = 4$, and $I_4 = 3$.

3. One legal schedule preserving the original dependences is:

Cycle	Memory	ALU	MUL
1	I_1		
2			
3		I_2	
4		I_3	
5			I_4

   If the compiler has renamed away the false dependences, I_3 could be scheduled earlier.

4. The encoding emits only real operations. The load row is one multi-op. The add and multiply row is one multi-op containing two operations. The empty row is represented by a pause field after the preceding multi-op. The store row is the next multi-op. Tail bits mark which operations end each multi-op, so unused slots do not need explicit no-op instructions.

5. $$ResII = \max\left(\left\lceil\frac{3}{1}\right\rceil, \left\lceil\frac{1}{1}\right\rceil, \left\lceil\frac{3}{1}\right\rceil\right) = 3.$$

   A loop-carried recurrence can still force MII higher if $\lceil Latency/Distance\rceil > 3$.

6. $$RecII = \left\lceil\frac{4}{1}\right\rceil = 4, \qquad MII = \max(ResII, RecII) = \max(3, 4) = 4.$$

7. A modulo reservation table has II rows because the steady-state schedule repeats every II cycles. Mapping unrolled cycle x to row $x \bmod II$ shows which operations from different iterations compete for the same recurring resource slot.

8. IMS begins at MII because MII is a proven lower bound. Trying smaller values wastes compile time because no legal schedule can satisfy the resource and recurrence constraints below that bound.

9. The prologue starts the first few overlapped iterations and fills the software pipeline. The kernel is the repeated steady-state schedule that launches a new iteration every II cycles. The epilogue drains remaining operations from the final iterations after no new iterations are started.

10. A compiler may move a load ahead of an older store only when it can prove the store and load do not access the same location, or when it has a correct speculation and recovery/checking mechanism. This is memory disambiguation because the legality depends on address aliasing and program memory order, not only on functional-unit availability.

8 Chapter 8: Multiprocessors and Coherence

1. SIMD uses one instruction stream over multiple data elements. MIMD allows multiple independent instruction streams over multiple data streams. A multicore with SIMD inside each core is classified overall as MIMD because the cores can execute different instruction streams, even though each core may contain SIMD lanes.

2. Shared memory gives programmers a single address-space abstraction and natural sharing, but it requires coherence, synchronization, and consistency rules. Message passing makes communication explicit and can scale well, but the programmer must manage data movement and communication structure directly.

3. A shared bus naturally broadcasts every transaction to all caches, so snooping controllers can observe requests. Point-to-point links such as PCIe do not automatically expose every request to every cache, so broadcast snooping is not the natural mechanism.

4. Without coherence, P_2 may hold or read a stale copy of $x = 1$. Its `x++` can therefore compute 2 even though other processors have written 22 and 44 elsewhere.

5. Coherence requires eventual write propagation, agreement on the order of writes to each location, and a processor reading back its own write when no intervening write changes that location. These per-location requirements do not establish a global program order across different locations. Propagation must allow "sufficient time" because real systems cannot make an update physically visible everywhere instantaneously.

6. In MSI, `M` means this cache has the only valid copy and it is dirty relative to memory. `S` means the cache has a clean readable copy that may also exist in other caches, and memory is up to date. `I` means the cache's copy is invalid and cannot be used.

7. One MSI trace is:

Step	$P1$	$P2$	$P3$	Memory event
$(P1, R)$	S	I	I	read
$(P1, W)$	M	I	I	none
$(P2, R)$	S	S	I	dirty writeback/flush
$(P3, R)$	S	S	S	read
$(P2, W)$	I	M	I	none; invalidations
$(P1, R)$	S	S	I	dirty writeback/flush
$(P3, W)$	I	I	M	read plus invalidations

8. In MESI, the first $P1$ read can place the line in `E` rather than `S` because no other cache has it. The later $P1$ write can then silently change `E`→`M` without a bus upgrade. After another cache reads the block, the behavior converges with the shared-state MSI behavior.

9. Starting with both caches in `S`, a write hit issues an upgrade or `GetM`. The writing cache moves to `M`; the other cache invalidates its copy and moves to `I`.

10. `IM` means the cache started in `I`, requested writable ownership, and is waiting for data and/or invalidation acknowledgments before it can enter `M`. A write miss that sends `GetM` enters `IM`.

11. The `E` state lets a cache that is the only clean holder later write the block silently. The protocol needs a way to detect whether other sharers exist because a read miss should receive `E` only when no other cache has the block.

12. In MSI, a cache in `M` that snoops `GetS` supplies or flushes the data, writes memory back to a clean value, and downgrades to `S`. In MOESI, it can move to `O`, supply the requester, and keep responsibility for the dirty shared value while memory remains stale. Memory is updated later when the owned dirty value is written back or ownership changes in a way that requires it.

13. The cache agent handles local processor `LOAD`/`STORE` requests and local cache-state transitions. The directory controller tracks global sharers/owners, orders coherence requests, sends invalidations or recalls, and waits for acknowledgments.

14. If all other copies are shared, `GetM` requires invalidating those sharers and waiting for acknowledgments. If another cache has the dirty copy, the directory must recall or forward the dirty data from that owner before granting writable ownership to the requester.

15. The directory is a sharer tracker because it records who may have the block. It is also a write-ordering point because it serializes grants of writable ownership, enforcing the single-writer invariant.

16. A controller-oriented MESI design may send `GETX` and enter `EM` so the directory learns that the cache is moving from exclusive clean ownership to dirty ownership. This keeps directory state and simulator counters consistent even when the data transfer itself is unnecessary.

17. The number of entries is
$$\frac{128\text{ GiB}}{64\text{ B}} = \frac{2^{37}}{2^6} = 2^{31}.$$
A full bit-vector entry stores $16 + 2 = 18$ bits, so total metadata is
$$18 \cdot 2^{31} = 38{,}654{,}705{,}664 \text{ bits} = 4{,}831{,}838{,}208 \text{ bytes} \approx 4.5 \text{ GiB}.$$

18. Each sharer ID needs $\lceil \log_2 16 \rceil = 4$ bits, so limited-directory sharer tracking uses $4K + 1$ bits including the overflow bit. Requiring $4K + 1 < 16$ gives $K \leq 3$. This performs poorly for widely shared read-mostly or migratory blocks that exceed the explicit sharer limit often.

19. Case (a), with X and Y in the same block, causes more invalidations and cache-to-cache movement because writes to Y also disturb processors using X. Case (b), with X and Y in different blocks, better avoids false sharing.

20. For $(P0, R)$, the cache misses and memory-read count increases. For $(P0, W)$, an `E`$\rightarrow$`M` silent-upgrade-style event may increase, with no memory access. For $(P1, R)$, $P0$'s dirty copy intervenes, so cache-to-cache transfer increases and $P0$ becomes owner/shared under MOESI-like behavior. For $(P0, \text{evict})$, the owned dirty value must be written back, so memory writes increase.

21. For many coherence questions, state and message behavior determines correctness and performance counters. Actual payload values are unnecessary unless the question asks for program-level computed data results.

22. One shared lower-level cache becomes a bandwidth and port-contention bottleneck. Even if it is coherent, all cores compete for the same access path and storage banks.

23. Slicing distributes blocks and directory entries across multiple banks/controllers. This increases parallel access bandwidth and reduces the central hot spot compared with one shared lower-level cache and one centralized directory.

24. Shared-memory summation needs a lock because multiple threads update the same `globalSum` variable. Message passing can give each worker a private partial sum and send results to a collector, avoiding simultaneous writes to one shared location.

25. Repeated test-and-set causes more coherence traffic because every poll is a write-like atomic operation that tries to take ownership. Test-and-test-and-set spins mostly on reads while the lock is held and attempts the write only when the value appears free.

26. Coherence makes individual memory locations behave consistently, but it does not provide mutual exclusion, atomic multi-step updates, phase ordering, or higher-level happens-before relationships. Locks and barriers provide those coordination properties.

9 Chapter 9: Synchronization, Transactions, and Consistency

1. States name stable conditions, but the protocol is the full set of request, response, transition, and ordering rules. For example, a block in `M` requested by another cache might be handled by abort/retry, intervention, ownership transfer, or writeback depending on the protocol.

2. Abort/retry tells the requester to try later while the dirty owner resolves the block. Intervene forwards the dirty data directly from the owner to the requester. The `O` state can delay write-back the longest because a cache may keep dirty shared ownership while supplying other sharers, leaving memory stale until a later eviction or ownership change.

3. Repeated `test_and_set()` performs write-like atomic operations on every spin attempt, repeatedly requesting exclusive ownership and invalidating other copies. Test-and-test-and-set spins with ordinary reads while the lock is busy and uses the atomic write only when the lock appears free.

4. Load-link records a reservation on a memory location. Store-conditional succeeds only if the reservation is still valid. A conflicting write or invalidating coherence event to the reserved block causes the store-conditional to fail. An ISA can also permit reservation loss or spurious failure for other reasons, so failure alone does not prove that the value changed.

5. Fetch-and-op atomically applies a general operation to a memory location and returns the old value. Fetch-and-add is the addition-specific case. It is useful for dynamic loop scheduling because each worker can atomically claim a unique chunk or index.

6. A tree lock spreads contention across several smaller locks so not all threads repeatedly invalidate the same global lock variable. The cost is extra acquisition latency because a thread may have to win locks at multiple levels.

7. A phase or toggle bit distinguishes the current barrier instance from the next one. Without it, a fast thread could observe a stale completion count from the previous barrier and pass through incorrectly.

8. A software counter barrier uses atomic memory operations and coherence traffic. A hardware barrier with one signal per PE assumes a shared observable signaling structure or dedicated wires that can combine arrivals directly. That assumption is much less natural in large point-to-point systems.

9. Use a lock to protect a critical section where only one thread may execute at a time, such as updating a shared total. Use a barrier when all threads must finish a phase before any thread starts the next phase, such as between bulk-synchronous simulation steps.

10. Transactional memory tries to avoid explicit lock ownership and some lock-induced serialization by executing critical sections optimistically and committing only if conflicts do not occur. A transaction may abort because another thread conflicts with its read/write set, because capacity is exceeded, because an unsupported instruction occurs, or because an interrupt or exception intervenes.

11. Cache coherence concerns the behavior of individual memory locations, especially propagation and serialization of writes to one block. Memory consistency defines the allowed ordering of operations across different locations as observed by threads.

12. If the producer's store to `flag` becomes visible before its earlier store to `x`, the consumer can read `flag == 1` and then still read an old `x`. Each location may be coherent, but the cross-location ordering is insufficient.

13. Under strict consistency with actual-time order $A1, B1, A2, B2, A3, B3$, A reads 20 and stores 24 to `0x3004`; B reads 10 and stores 19 to `0x3008`. Final values are

$$M[\texttt{0x3008}] = 19, \qquad M[\texttt{0x3004}] = 24.$$

    A legal sequentially consistent interleaving $A1, A2, A3, B1, B2, B3$ gives

$$M[\texttt{0x3008}] = 33, \qquad M[\texttt{0x3004}] = 24.$$

    Another legal sequentially consistent interleaving $B1, B2, B3, A1, A2, A3$ gives

$$M[\texttt{0x3008}] = 19, \qquad M[\texttt{0x3004}] = 23.$$

14. Sequential consistency guarantees that the result is as if all operations executed in one global interleaving that preserves each processor's program order. It is usually more expensive than acquire/release models because it restricts reordering and buffering even when synchronization-specific ordering would be enough.

15. A data race occurs when two threads access the same memory location, at least one access is a write, and there is no synchronization ordering between them. Coherence still allows races because coherence orders writes to a location but does not make a multi-operation program protocol atomic or properly ordered.

16. Strict consistency ties visibility directly to physical time: a read sees the most recent write in real time. Sequential consistency only requires one global order that preserves each processor's program order, not actual physical-time order.

17. For both $AA = 0$ and $BB = 0$, each processor's read would need to occur before the other processor's write. But each processor's program order places its own write before its read. Those four ordering requirements form a cycle, so no sequentially consistent interleaving can produce both reads as 0.

18. In a minimal teaching model, a store-ordering fence orders earlier stores before later stores, and a load-ordering fence orders earlier loads before later loads. A full memory fence orders the specified earlier loads and stores before the specified later loads and stores. Exact predecessor/successor guarantees depend on the ISA; fence names alone do not establish every cross-type ordering. Fences are expensive because they can drain store buffers, block speculation, and stall memory-level parallelism.

19. Read-only shared data creates no invalidations and no write-write or read-write races after initialization. Locks and atomics are more delicate because their correctness depends on both visibility and ordering of writes across locations.

10 Chapter 10: Parallel Interconnection Networks

1. A parallel interconnection network is part of the machine's internal communication fabric, so its latency, bandwidth, routing, and flow control directly shape processor and memory-system behavior. Flit width matters because it determines how much of a packet can move per link cycle and affects serialization latency, buffer sizing, and switch datapath width.

2. In a direct topology, processors or routers attached to processors are the vertices of the graph. In an indirect topology, communication passes through separate switch stages, so the processors are endpoints rather than the internal switch vertices.

3. $64 = 2^6$, so the hypercube has dimension 6. Each node has degree 6.

4. k-ary n-dimensional means there are n dimensions and k positions in each dimension. A 4-ary 3-dimensional network has
$$4^3 = 64$$
nodes.

5. A mesh has edges that stop at the boundary. A torus adds wraparound links. The torus usually has higher bisection bandwidth because a dimension-aligned cut crosses links on both sides of the wraparound dimension.

6. From node 0 to node 5 in a unidirectional ring is 5 hops, so zero-contention link latency is 5 cycles. If one older message is already queued at node 5 and the node consumes one message per cycle, the arriving message sees one extra cycle of service delay.

7. A logical ring can require long wraparound or nonlocal wires on a planar chip or board. Those wires can increase delay, area, energy, and routing congestion even though the graph is logically simple.

8. Bisection bandwidth is the minimum total link bandwidth crossing a cut that divides the network into two equal halves. A 4-ary 2-mesh has one set of four links across a dimension-aligned bisection. The corresponding torus has wraparound links crossing the cut as well, giving twice the crossing bandwidth with the same per-link bandwidth.

9. A dimension-aligned equal cut attains the minimum in the regular square mesh and torus examples and makes their crossing links easy to count. Other equal-halves partitions are still included in the definition. A partition with more crossing links cannot raise the minimum bisection bandwidth.

10. A full crossbar can connect many input-output pairs directly but grows very quickly in hardware cost. A multi-stage indirect network is cheaper and more regular, even though some input-output combinations can block each other.

11. A perfect shuffle permutes wires by rotating or rearranging endpoint bits between stages. Omega networks use perfect-shuffle-style interstage wiring so that each stage consumes one destination bit and progressively routes packets toward the selected output.

12. An omega network with 16 endpoints needs

$$\log_2 16 = 4$$

stages, and one route consumes 4 destination bits.

13. Deterministic routing always chooses the same path for a given source and destination. Adaptive routing responds to congestion or faults. Oblivious routing uses randomized or traffic-independent choices without observing current congestion. Valiant-style routing tries to spread traffic and avoid hot spots by routing through an intermediate destination.

14. A packet is the full logical message unit handled by the network. A flit is a smaller flow-control unit of that packet that moves through links and buffers.

15. If the desired downstream buffer or output is full, the router can block/stall the packet, buffer it locally, or route it through another legal path if adaptive routing is available. Some systems may also drop/retry, but lossless on-chip networks usually avoid dropping.

16. Store-and-forward receives a whole packet before forwarding it. Cut-through can begin forwarding after the header selects the route. Wormhole divides packets into flits that occupy buffers and links along the path. Virtual channels provide multiple logical queues over one physical channel and most directly address head-of-line blocking.

17. Virtual channels require extra per-channel buffers, allocation logic, flow-control state, and arbitration. The router must choose among multiple logical queues while preserving correctness and avoiding deadlock.